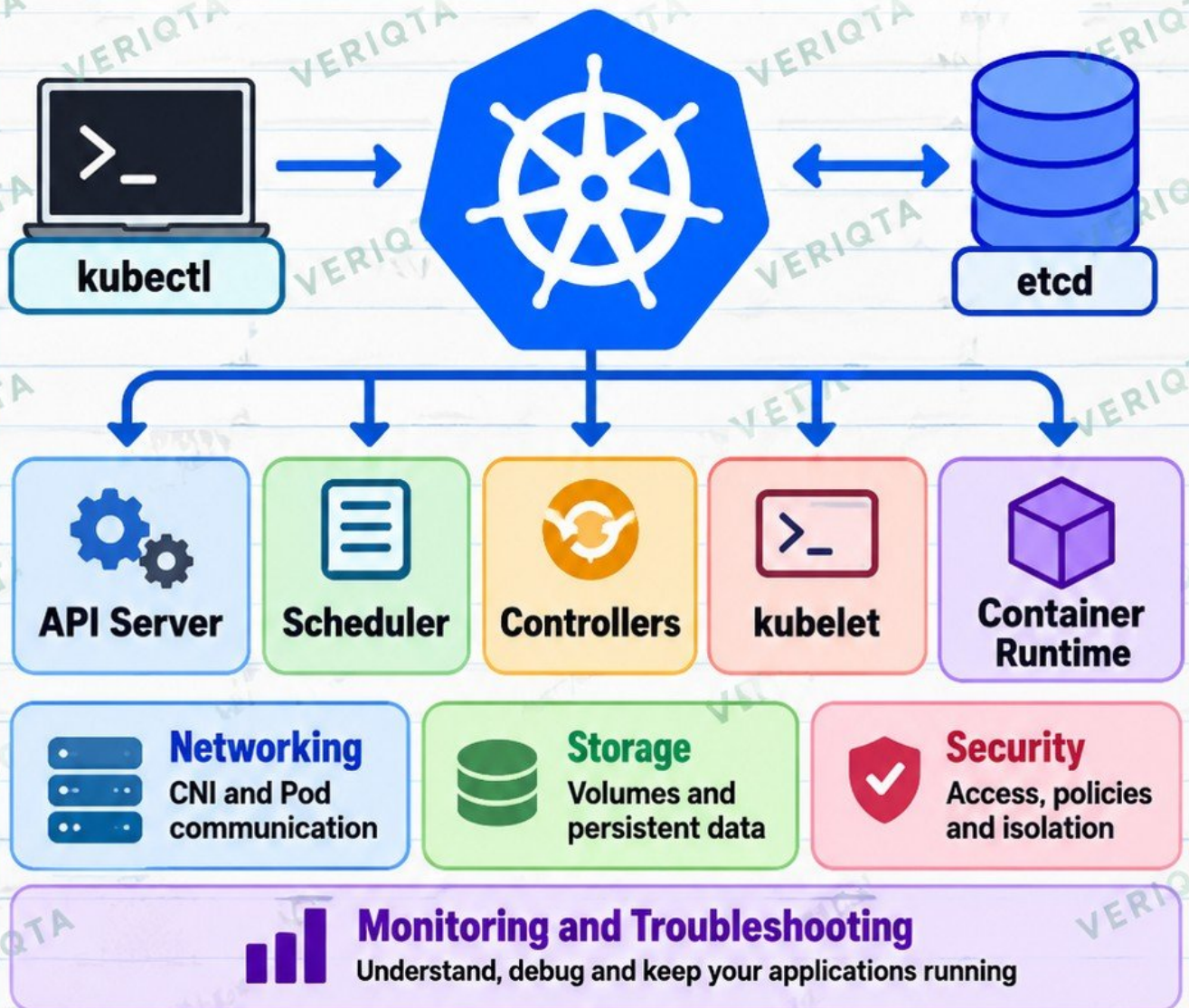


KUBERNETES

UNDER THE HOOD

How Kubernetes Really Works From Your Command to a Running Application



@veriqta



@veriqta



@veriqta



@veriqta



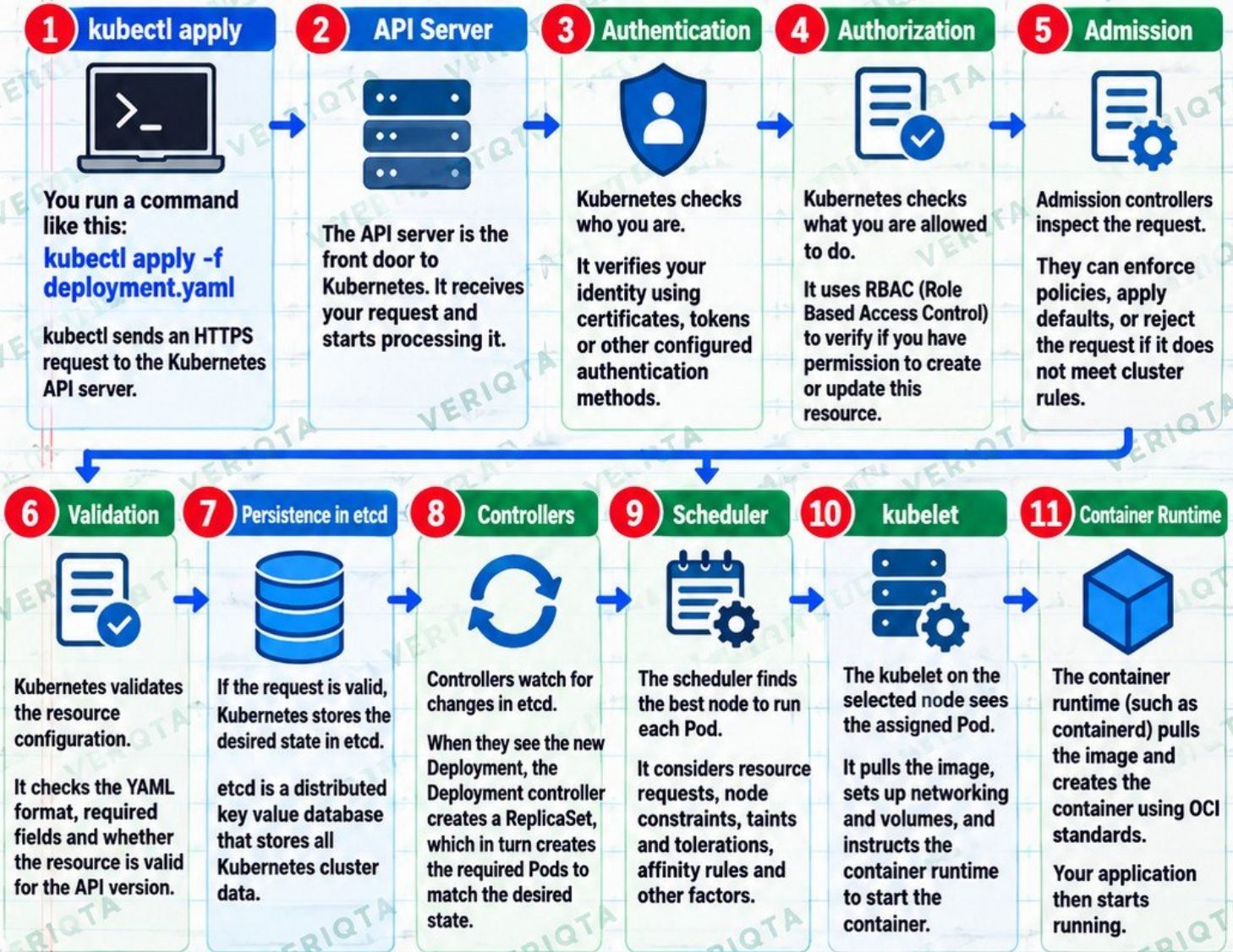
@veriqta



@veriqta

What Really Happens When You Run `kubectl apply`?

When you run `kubectl apply -f deployment.yaml`, your request goes through multiple Kubernetes components before your application is running. This page follows one request step by step so you understand what is happening under the hood.



Key Takeaways

- ✓ kubectl does not create containers directly.
- ✓ The API server stores your desired state in etcd.
- ✓ Controllers, scheduler, kubelet and the container runtime work together to make your application run.
- ✓ Kubernetes continually watches and reconciles the state to match what you defined.

Why This Matters

- Helps you understand where to look when something goes wrong.
- Explains why changes in Kubernetes can take a few seconds to appear.
- Shows how multiple components work together.
- Builds the mental model for debugging and operating real Kubernetes clusters.

Real World Example

You deploy a web application:

`kubectl apply -f deployment.yaml`

Within a few seconds, the Deployment controller creates the required Pods, the scheduler assigns them to nodes, the kubelet starts the containers and your application is running and accessible through a Service.



@veriqta



@veriqta



@veriqta



@veriqta



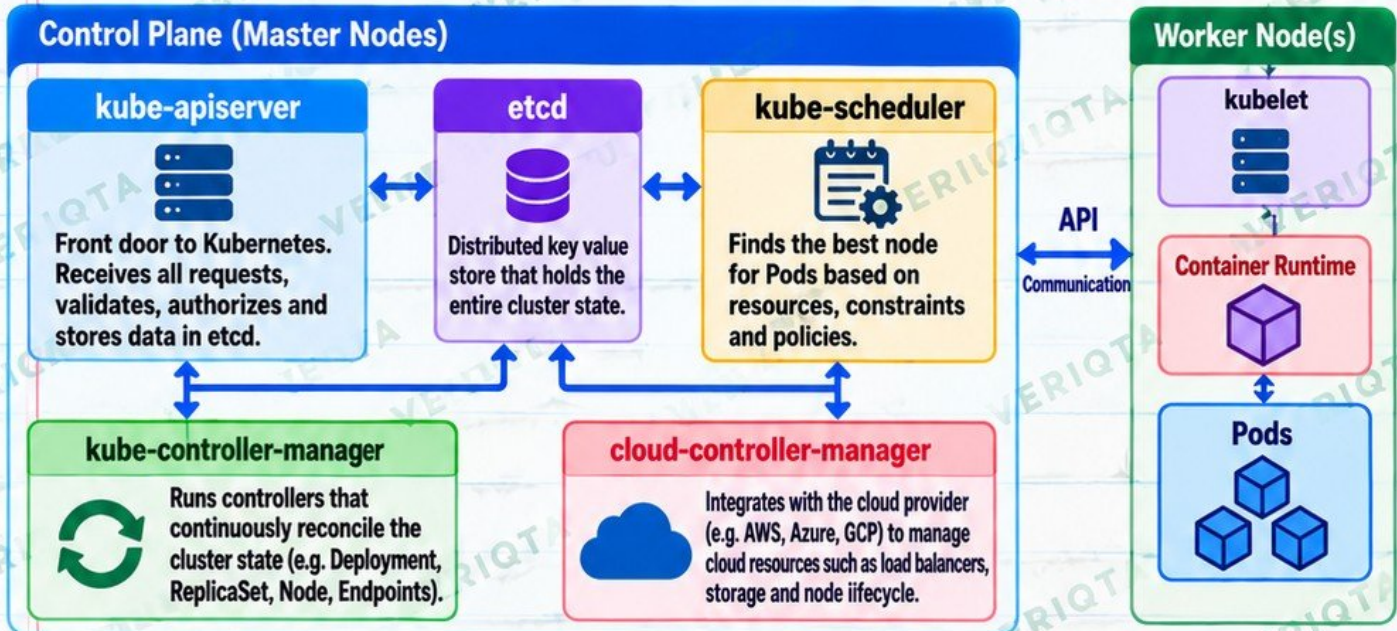
@veriqta



@veriqta

The Kubernetes Control Plane

The Kubernetes control plane is a set of components that work together to manage the cluster. Kubernetes is not one program making every decision. Each component has a specific responsibility, and they communicate through the API server.



1 kube-apiserver

- Central entry point for all clients and components.
- Receives requests through the Kubernetes API.
- Handles authentication, authorization and admission.
- Validates the request and stores the desired state in etcd.
- All other components (scheduler, controllers, kubelet) communicate with the API server, not directly with etcd.

2 etcd

- A distributed key value database.
- Stores the entire cluster state (all Kubernetes objects).
- Highly available and consistent.
- If etcd is unavailable, the cluster cannot make or store changes.
- Stores desired state, not your application data.

3 kube-scheduler

- Watches for new Pods that do not have a node assigned.
- Evaluates all available nodes using filters and scoring.
- Considers resource requests, node constraints, taints, tolerations, affinity rules and other factors.
- Assigns the Pod to the most suitable node.
- Writes the scheduling decision back to the API server.

4 kube-controller-manager

- Runs multiple controllers in a continuous loop.
- Watches the cluster state and takes action to match the desired state.
- Examples: Deployment controller, ReplicaSet controller, Node controller, Endpoints controller, Service controller.
- If a Pod or node fails, controllers detect it and take corrective action.

5 cloud-controller-manager

- Integrates with your cloud provider (AWS, Azure, GCP, etc).
- Manages cloud specific resources such as load balancers, routes, storage and node lifecycle.
- Runs controllers that interact with the cloud provider APIs.
- Not needed for on-premises or self managed environments.



Key Takeaways

- The control plane is made up of multiple components.
- Each component has a specific responsibility.
- All components communicate through the API server.
- They work together to maintain the desired state of the cluster.
- This separation of responsibilities makes Kubernetes scalable, reliable and extensible.



@veriqta



@veriqta



@veriqta



@veriqta



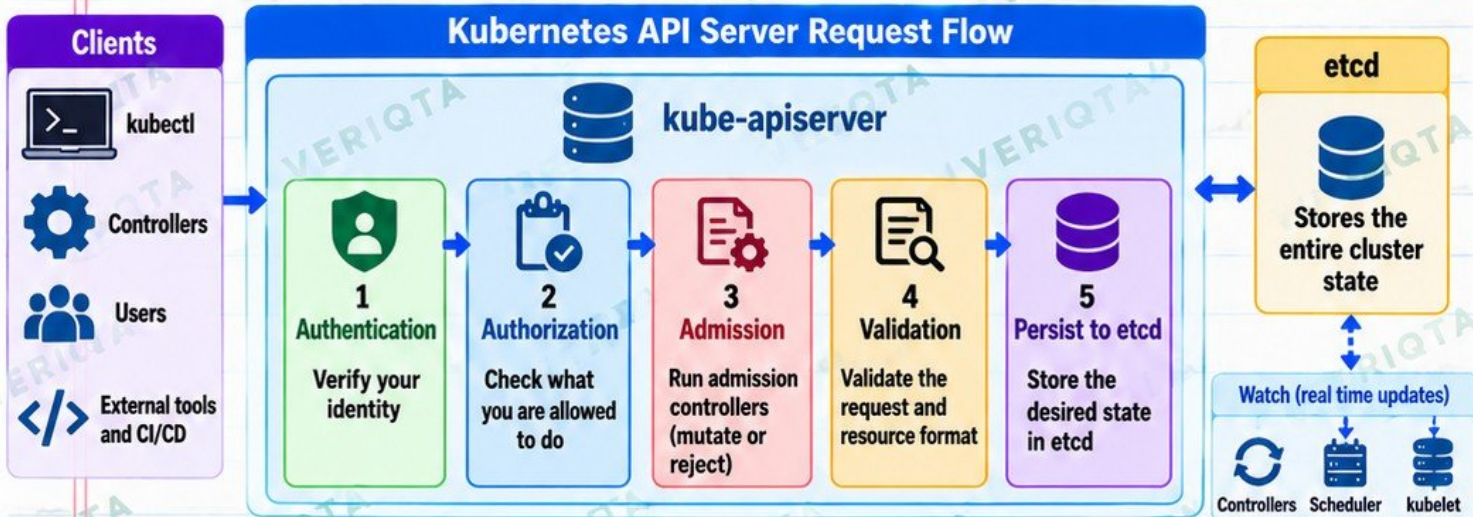
@veriqta



@veriqta

The API Server: Kubernetes' Front Door

The API server is the central entry point for all clients and components. It receives requests, authenticates and authorizes them, runs admission and validation, and persists the desired state in etcd. All Kubernetes components communicate through the API server. No component directly modifies etcd.



1 Authentication

- Verifies who you are.
- Supports client certificates, tokens (e.g. service accounts), and external identity providers (e.g. OIDC).
- If authentication fails, the request is rejected.

2 Authorization

- Checks what you are allowed to do.
- Uses RBAC (Role Based Access Control) or other authorization methods.
- Determines if you can create, read, update or delete a resource.

3 Admission Controllers

- Inspect and optionally modify the request before it is accepted.
- Can enforce policies, set defaults or reject invalid or non compliant requests.
- Examples: Namespace lifecycle, resource limits, security policies.

4 Validation

- Validates the request and the resource format (e.g. YAML/JSON).
- Checks required fields, data types and API version.
- If validation fails, the request is rejected with an error.

5 Persistence in etcd

- If the request is valid, the desired state is stored in etcd.
- etcd is a distributed key value database.
- It stores all Kubernetes objects (e.g. Pods, Deployments, Services, ConfigMaps, Secrets).

6 Watches and Notifications

- The API server sends real time updates (watches) to components that are interested in changes.
- Controllers, scheduler and kubelet watch for changes and take action.
- This is how Kubernetes reacts quickly to updates and failures.

Example API Request

```
kubectl apply -f deployment.yaml
```

This sends an HTTPS request to the API server to create or update the Deployment resource.

Why Components Use the API Server

- Provides a single, consistent source of truth (etcd).
- Enforces security, validation and policies.
- Prevents components from making conflicting changes.
- Enables audit logs, observability and reliability.

Key Takeaways

- The API server is the front door to Kubernetes.
- All clients and components communicate through it.
- It handles authentication, authorization, admission and validation.
- Valid requests are stored in etcd.
- Other components watch for changes and reconcile the desired state.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

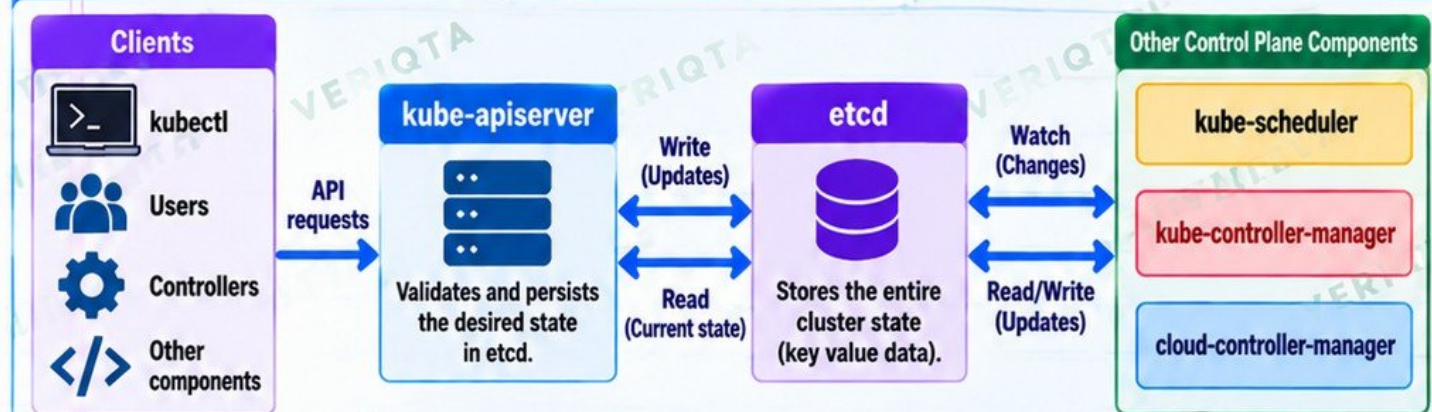


@veriqta

etcd: Where Cluster State Lives

etcd is a distributed key value database that stores the entire Kubernetes cluster state. Kubernetes uses etcd to keep the desired state of all objects such as Pods, Deployments, Services, ConfigMaps, Secrets and more. It is critical for the control plane to work.

Where etcd Fits in Kubernetes



1 What etcd Stores

- All Kubernetes objects (Pods, Deployments, Services, ConfigMaps, Secrets, Namespaces, etc).
- Cluster configuration and state.
- Resource metadata, specifications and status.
- Flags, settings and other internal Kubernetes data.
- It stores the **desired state**, not your application data.

2 Key Value Structure

Key (Path)	Value (Example)
/registry/pods/default/nginx-xyz	Pod object (YAML/JSON)
/registry/deployments/default/nginx	Deployment object
/registry/services/default/web	Service object
/registry/configmaps/default/app	ConfigMap object
/registry/secrets/default/db	Secret object

3 Consistency and High Availability

- etcd uses the Raft consensus algorithm.
- Data is replicated across multiple etcd nodes (usually 3 or 5).
- A quorum is required to make changes.
- This ensures consistency and prevents data loss.

4 Why etcd is Critical

- The entire Kubernetes cluster depends on it.
- If etcd is unavailable, the API server cannot read or store changes.
- No new objects can be created or updated.
- Controllers and the scheduler cannot see the current state.
- The cluster becomes read only or completely unavailable.

5 If etcd Becomes Unavailable

- kubectl commands will fail or time out.
- No new changes can be made to the cluster.
- Controllers cannot detect or react to changes.
- The scheduler cannot schedule new Pods.
- Running Pods may continue to run, but the cluster state cannot be updated.

6 Backup and Restore Awareness

- Regularly back up etcd.
- Use etcd snapshot for backups.
- Store backups securely.
- Test the restore process.
- In a disaster, restore etcd and then restart the control plane components.

7 Important Distinction

- ✗ **Cluster State (in etcd)**
 - Kubernetes objects and configuration
 - Desired state of the cluster

- ✓ **Application Data (not in etcd)**
 - Your application data (databases, files, user data, etc) is NOT stored in etcd.
 - Use persistent storage solutions (e.g. PVC, cloud storage, external DB).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

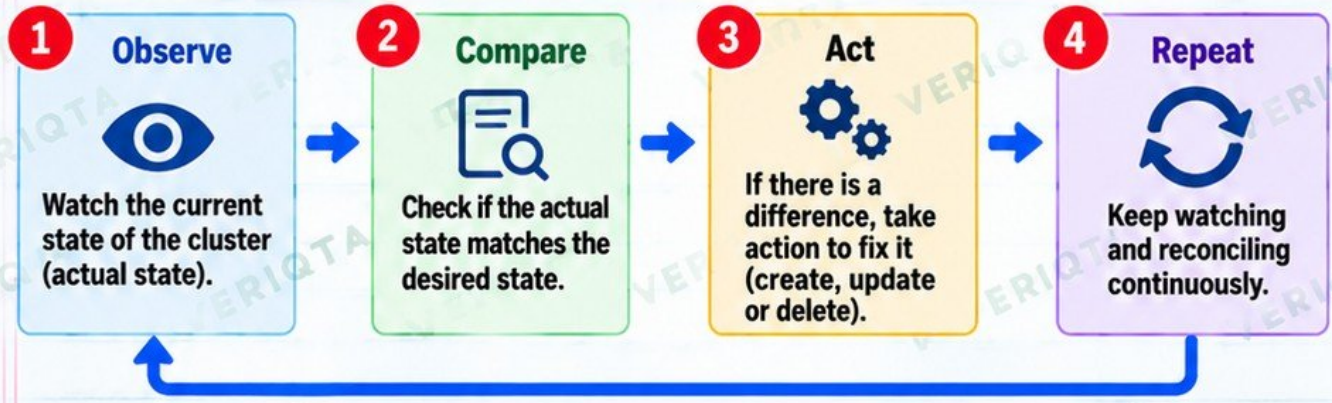


@veriqta

Desired State and Reconciliation

Kubernetes uses a reconciliation loop to continuously make the actual state match the desired state. You declare what you want, and Kubernetes keeps working to ensure the cluster matches it, even when things change or fail.

The Reconciliation Loop



1 Desired vs Actual State

- Desired state is defined in your YAML files (what you want).
- Actual state is the real state in the cluster (what is currently running).
- Kubernetes keeps comparing both and makes changes until they match.

2 Reconciliation Loop

- Controllers continuously watch for changes in the cluster.
- When they see a difference between desired and actual state, they take action.
- This loop runs continuously, not just once.

3 Eventual Convergence

- Kubernetes keeps working until the actual state matches the desired state.
- If a Pod is deleted or a node fails, Kubernetes automatically recreates or reschedules it.
- This is why your application recovers without manual intervention.

4 Declarative Configuration

- You declare the desired state using YAML (e.g. Deployment with 3 replicas).
- You do not tell Kubernetes exactly how to do it.
- Kubernetes figures out the required steps to reach and maintain that state.

5 Why Kubernetes Continually Works

- The cluster is dynamic and can change at any time (nodes, Pods, network, etc).
- Controllers ensure the desired state is always maintained.
- It is not a one time execution.
- This design makes the system self healing and resilient.

6 Real World Example

- You define a Deployment with 3 replicas.
- One Pod is deleted or crashes.
- The controller detects the difference (actual: 2, desired: 3) and creates a new Pod automatically.
- The cluster returns to the desired state without manual action.

> Example: Deployment Reconciliation

Desired State (YAML)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  
```

Actual State
(Only 2 Pods running)



Reconciles
(Creates a new Pod)



Key Takeaways

- Kubernetes uses reconciliation loops.
- It continually compares desired and actual state.
- Controllers take action to fix any difference.
- The system eventually converges to the desired state.
- You declare what you want, not how to do it.
- This makes Kubernetes reliable, self healing and scalable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

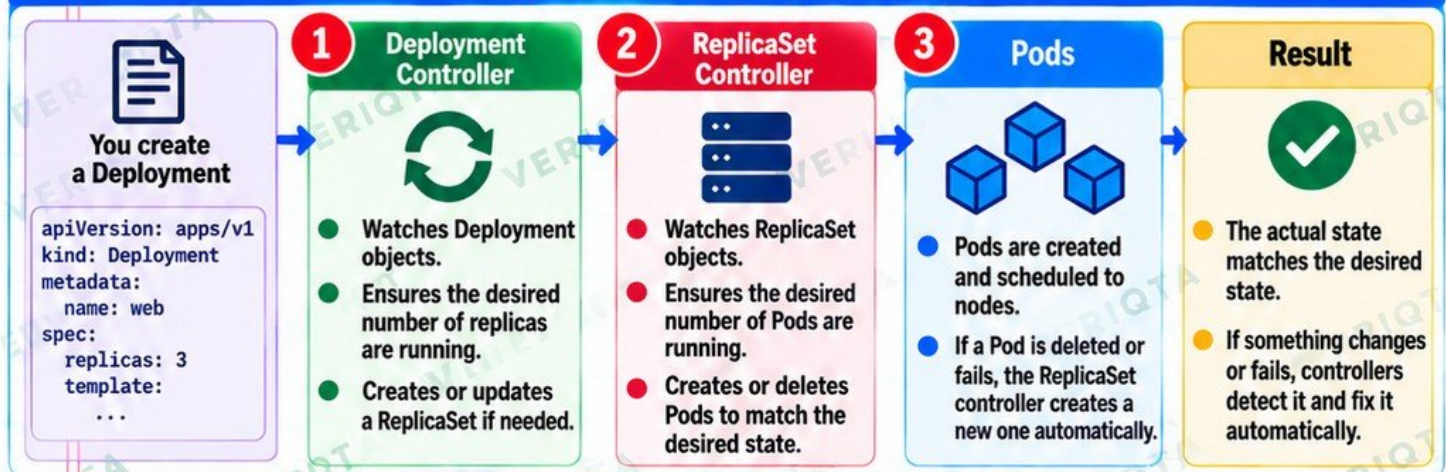


@veriqta

Controllers: The Automation Behind Kubernetes

Controllers watch Kubernetes resources, compare the actual state with the desired state and take action to make them match. They run continuously and are responsible for keeping the cluster in the desired state.

How a Deployment is Managed by Controllers



1 What is a Controller?

- A controller is a control loop that watches resources and reconciles state.
- It compares the actual state with the desired state and takes action to make them match.
- Controllers run continuously inside the kube-controller-manager.

2 Common Controllers

- Deployment controller
- ReplicaSet controller
- StatefulSet controller
- DaemonSet controller
- Job controller
- Node controller
- Endpoint controller
- and many others...

3 Controller Responsibility

- Monitor the current state of resources.
- Take action when the actual state does not match the desired state.
- Create, update or delete resources.
- Handle failures and maintain the required number of replicas.
- Ensure high availability and self healing.

4 Ownership and ownerReferences

- When a controller creates a resource, it sets the ownerReferences field.
- This tells Kubernetes which object owns the resource.
- If you delete the owner (e.g. Deployment), Kubernetes also deletes the owned resources (e.g. ReplicaSet and Pods).
- This maintains proper cleanup and prevents orphaned resources.

5 Why Deleting or Changing Triggers Automatic Replacement

- Controllers are always watching for changes.
- If a Pod is deleted, the ReplicaSet controller detects it and creates a new one.
- If you update the Deployment (e.g. new image), the Deployment controller creates a new ReplicaSet and gradually replaces the Pods (rolling update).
- If a node fails, the controller ensures the required number of Pods are still running.

6 Real World Example

- You have a Deployment with 3 replicas.
- One Pod crashes.
- The ReplicaSet controller detects the mismatch (desired 3, actual 2).
- It creates a new Pod automatically.
- Your application remains available without manual intervention.

> Check the Relationship

```

# View the deployment
kubectl get deployment web

# View the replicaset created by the deployment
kubectl get rs

# View the owner reference
kubectl get rs -o yaml | grep -A 5 ownerReferences
  
```

Key Takeaways

- ✓ Controllers keep the cluster in the desired state. They watch, compare and take action continuously.
- ✓ Deployment → ReplicaSet → Pods is a common example.
- ✓ ownerReferences link resources and enable automatic cleanup.
- ✓ Deleting or changing resources can trigger automatic replacement.
- ✓ Controllers make Kubernetes reliable, self healing and scalable.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

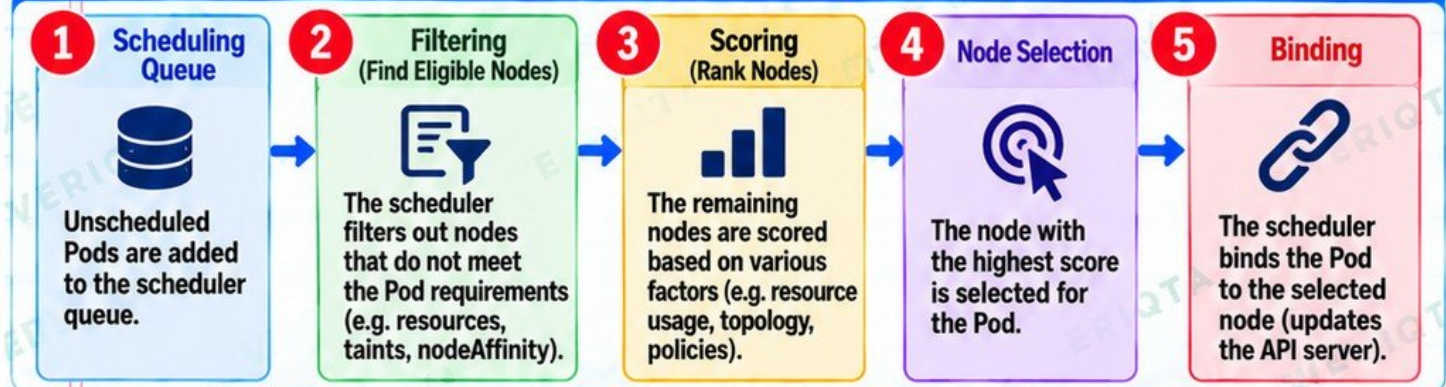


@veriqta

How the Scheduler Chooses a Node

The Kubernetes scheduler watches for unscheduled Pods and selects the best node for each Pod based on resource requirements, constraints and policies. It filters nodes, scores the remaining nodes, chooses one and binds the Pod to that node.

The Scheduling Flow (Step by Step)



- 1 Scheduling Queue**
 - The scheduler watches the API server for Pods without a node (Pending).
 - These Pods are added to an internal queue.
 - They are processed one at a time or in batches.
 - Pods can wait in the queue if no suitable node is available.
- 2 Filtering (Predicates)**
 - Nodes are filtered based on hard requirements.
 - Checks include: available CPU and memory, taints and tolerations, node selectors, node affinity, required topology constraints, and other policies.
 - Nodes that do not meet the requirements are removed from the list.
- 3 Scoring (Priorities)**
 - The remaining nodes are scored using multiple criteria.
 - Factors include: best resource fit, spread across zones, least resource usage, data locality, and custom scheduling policies.
 - Each node gets a score, and the highest scoring node is chosen.
- 4 Node Selection and Binding**
 - The node with the highest score is selected.
 - The scheduler creates a binding request to the API server.
 - The Pod is then assigned to the node and the Pod status is updated.
 - The kubelet on the selected node picks it up and starts the containers.
- 5 Resource Requests and Limits**
 - The scheduler uses resource requests (not limits) to make decisions.
 - If a node does not have enough available CPU or memory, it will be filtered out.
 - Requests must match the available allocatable resources on the node.
 - Limits do not affect scheduling decisions.
- 6 Taints and Tolerations**
 - Taints allow nodes to reject Pods that do not have a matching toleration.
 - Common use cases: dedicated nodes, special hardware, critical workloads.
 - A Pod must have a matching toleration to be scheduled on a tainted node.
- 7 Node Affinity**
 - Node affinity allows you to prefer or require specific nodes based on labels.
 - Types: `requiredDuringSchedulingIgnoredDuringExecution` (hard) and `preferredDuringSchedulingIgnoredDuringExecution` (soft).
 - Uses labels on nodes (e.g. `disktype=ssd`).
 - Helps place Pods on the right nodes.
- 8 Topology Constraints**
 - `TopologySpreadConstraints` help spread Pods across zones, nodes or other topology domains.
 - Prevents all replicas from being scheduled on the same node or zone.
 - Improves availability and resilience.
- 9 Why a Pod Can Remain Pending**
 - Not enough CPU or memory on any node.
 - Node taints without matching tolerations.
 - Node affinity or node selector not matching.
 - Topology constraints cannot be satisfied.
 - `PersistentVolumeClaim` not bound (in `WaitForFirstConsumer`).
 - All nodes are unschedulable or cordoned.
 - Other policies or constraints.

Key Takeaways

- The scheduler finds the best node by filtering and scoring.
- It uses resource requests, taints, tolerations, affinity and topology constraints.
- Only nodes that meet all requirements are considered.
- If no node is suitable, the Pod remains Pending.
- After binding, the kubelet on the node starts the Pod.
- Proper resource requests and constraints help with reliable scheduling.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

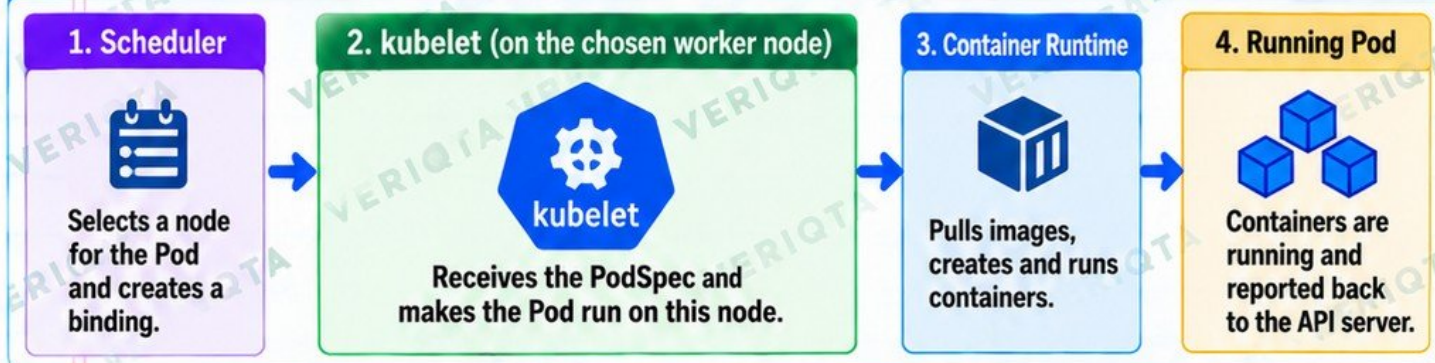


@veriqta

kubelet: What Happens on the Worker Node

The kubelet is the primary node agent that runs on every worker node. After a Pod is assigned to its node by the scheduler, the kubelet takes over and makes the Pod run. It works with the container runtime, sets up volumes and networking, manages container lifecycle, runs probes, and continuously updates the Pod and node status.

Where kubelet Fits in the Flow



1 What is kubelet?

- A node agent that runs on every worker node.
- Receives PodSpecs from the API server.
- Makes sure the containers defined in the PodSpec are running.
- Reports Pod and node status back to the API server.
- Runs continuously as a control loop.

2 PodSpec Processing

- Watches for new Pods assigned to this node.
- Reads the PodSpec (containers, volumes, probes, etc).
- Pulls the required container images (if not present).
- Creates the containers using the container runtime.

3 Interaction with Container Runtime

- Uses the Container Runtime Interface (CRI) to communicate.
- Works with containerd, CRI-O or other CRI-compliant runtimes.
- Requests image pull, container creation, start, stop and delete.
- Monitors container health and status through the runtime.

4 Volume Setup

- Mounts volumes defined in the PodSpec (e.g. ConfigMaps, Secrets, PVCs, emptyDir).
- Works with the Container Storage Interface (CSI) for persistent volumes.
- Ensures volumes are available before starting the containers.
- Handles volume mount and unmount during Pod lifecycle.

5 Probes and Health Checks

- Runs liveness probes to check if the container is healthy.
- Runs readiness probes to check if the container is ready to receive traffic.
- Runs startup probes when configured.
- Takes action based on probe results (e.g. restart unhealthy containers).

6 Container Lifecycle

- Starts containers as defined in the PodSpec.
- Monitors container status (running, waiting, terminated).
- Restarts containers based on the restartPolicy (e.g. Always, OnFailure).
- Handles graceful termination when a Pod is deleted (sends SIGTERM, then SIGKILL if needed).

7 Node Status Reporting

- Reports node status (CPU, memory, disk, network, conditions) to the API server.
- Reports Pod status, container status and events.
- Updates status continuously, not just once.
- Helps the control plane understand the health of the node and Pods.

8 Scheduler vs kubelet

Scheduler (What Node?)	kubelet (Make it Run)
• Finds the best node for the Pod. • Considers resources, constraints and policies. • Creates a binding to the chosen node.	• Receives the PodSpec on the chosen node. • Pulls images and creates containers. • Sets up volumes and networking. • Runs and monitors the containers.

9 Key Takeaways

- The scheduler chooses a node, kubelet makes the Pod run there.
- kubelet works with the container runtime, storage and networking.
- It manages the full container lifecycle.
- It runs health checks and reports status.
- It continuously ensures the actual state matches the desired state from the API server and controllers.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

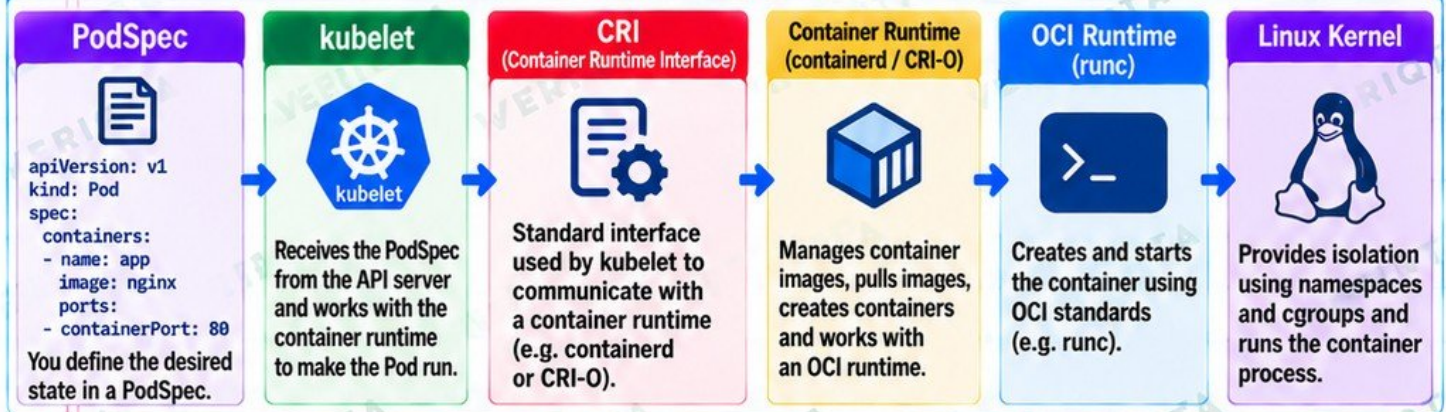


@veriqta

From PodSpec to Running Container

This shows what happens beneath kubelet after a Pod is scheduled to a node. Kubernetes does not directly run your application. It uses the kubelet, the Container Runtime Interface (CRI), a container runtime, and an OCI runtime to create and run containers on the Linux kernel.

The Complete Flow



1 What is the CRI?

- The Container Runtime Interface (CRI) is a standard interface for kubelet to communicate with a container runtime.
- It allows different runtimes (e.g. containerd, CRI-O) to be used with Kubernetes.
- Kubelet uses gRPC calls over the CRI to manage containers, images and sandboxes.
- CRI hides the runtime implementation details from kubelet.

2 Container Runtimes

- containerd is the most widely used runtime in Kubernetes.
- CRI-O is a lightweight runtime focused on Kubernetes.
- The runtime is responsible for image management, container lifecycle and storage.
- You can switch runtimes as long as they implement the CRI.

3 OCI Runtime and OCI

- The OCI (Open Container Initiative) defines standards for container images and runtimes.
- runc is the most common OCI runtime.
- The OCI runtime creates and runs the container using Linux kernel features (namespaces, cgroups, etc).
- This ensures containers run in a standard and secure way.

4 Image Pulling Process

- The runtime pulls the container image from the registry (e.g. Docker Hub, ECR).
- Images are stored locally on the node (in containerd/CRI-O).
- If the image is already present, it is taken from the local cache.
- Image pull policies (Always, IfNotPresent, Never) control when images are pulled.

5 Container Creation Steps

- kubelet sends a request through the CRI to create a container.
- The runtime creates a sandbox (network and isolation setup).
- The OCI runtime (runc) creates the container using Linux namespaces and cgroups.
- The container process (your application) is started inside the container.

6 Why Kubernetes Does Not Directly Execute Your Application

- Kubernetes is a control plane and orchestration system, not a process manager.
- It relies on the container runtime to run containers.
- This design provides standardization, isolation, security and flexibility.
- Kubernetes focuses on desired state, scaling, healing and management, not executing application processes.

7 Key Components Summary

Component	Role
kubelet	Node agent, works with CRI to run Pods
CRI	Standard interface between kubelet and runtime
containerd / CRI-O	Container runtime (manages images and containers)
runc (OCI runtime)	Creates and starts containers using OCI standards
Linux kernel	Provides isolation (namespaces, cgroups) and runs container processes

8 Key Takeaways

- ✓ kubelet does not run containers directly.
- ✓ It uses the CRI to communicate with a container runtime.
- ✓ containerd or CRI-O manages images and containers.
- ✓ runc (OCI runtime) creates and starts the container.
- ✓ The Linux kernel provides isolation and runs the container process.
- ✓ This layered design makes Kubernetes secure, flexible and portable across different runtimes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

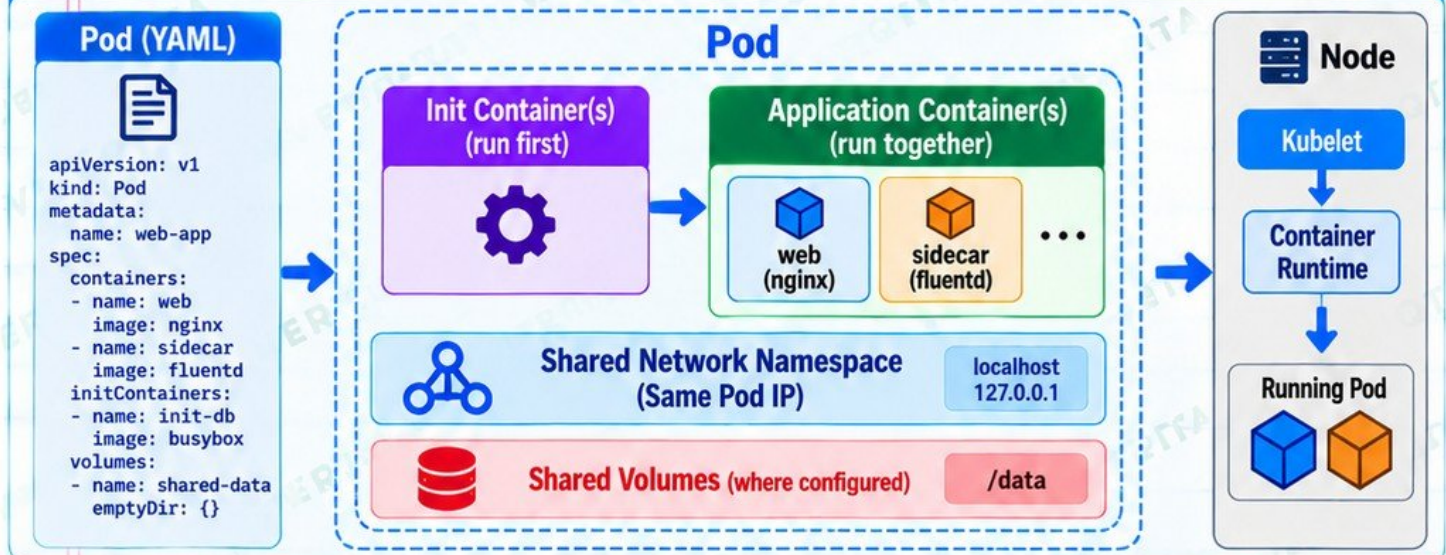


@veriqta

What a Pod Really Is

A Pod is more than just the “smallest deployable unit”. It is a shared environment for one or more containers that run together on the same node, share network and storage (where configured), and are tightly coupled.

Anatomy of a Pod



1 Pod Sandbox

- A Pod provides a shared execution environment for its containers.
- It includes a shared network namespace, shared IPC (by default) and shared storage (where configured).
- All containers in the Pod run on the same node.

2 Shared Network Namespace

- All containers in a Pod share the same network namespace.
- They use the same Pod IP address.
- Containers can communicate with each other using localhost (127.0.0.1) and do not need to use the Pod IP.
- This allows them to act as one application.

3 Pod IP

- Every Pod gets a unique IP address on the cluster network.
- The Pod IP is shared by all containers in the Pod.
- External traffic to the Pod IP can reach any container listening on a port.
- When the Pod is deleted, the Pod IP is also deleted (it is not permanent).

4 Shared Volumes

- Containers in the same Pod can share volumes (e.g. emptyDir, ConfigMap, Secret, PVC).
- This allows containers to read and write the same files.
- Volumes are defined in the PodSpec and mounted into the containers.

5 Multiple Containers

- A Pod can run one or more containers.
- Common pattern: main application container + sidecar container (e.g. logging, monitoring, proxy).
- All containers share the same network and can communicate using localhost.
- They are designed to work together as a single unit.

6 Init Containers

- Init containers run and complete before the main containers start.
- They are used for setup tasks (e.g. database initialization, config preparation, waiting for dependencies).
- If an init container fails, the main containers will not start.

7 Tightly Coupled Containers

- Containers in a Pod are tightly coupled to a single application or service.
- They share the same lifecycle (created, scheduled, started and stopped together).
- If one container in the Pod fails and cannot recover, the Pod may be restarted.
- Pods are not intended to run unrelated applications.

8 When to Use Multiple Containers in a Pod

- Sidecar patterns (e.g. logging, monitoring, service mesh proxy).
- Helper containers (e.g. configuration reloader).
- Init containers for setup tasks.
- Containers that must share the same network and storage and work very closely together.

9 Key Takeaways

- ✓ A Pod is a shared environment for one or more containers.
- ✓ All containers in a Pod share the same network namespace, Pod IP and volumes (where configured).
- ✓ Containers can communicate using localhost.
- ✓ Init containers run before the main containers.
- ✓ Containers in a Pod are tightly coupled and share the same lifecycle.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

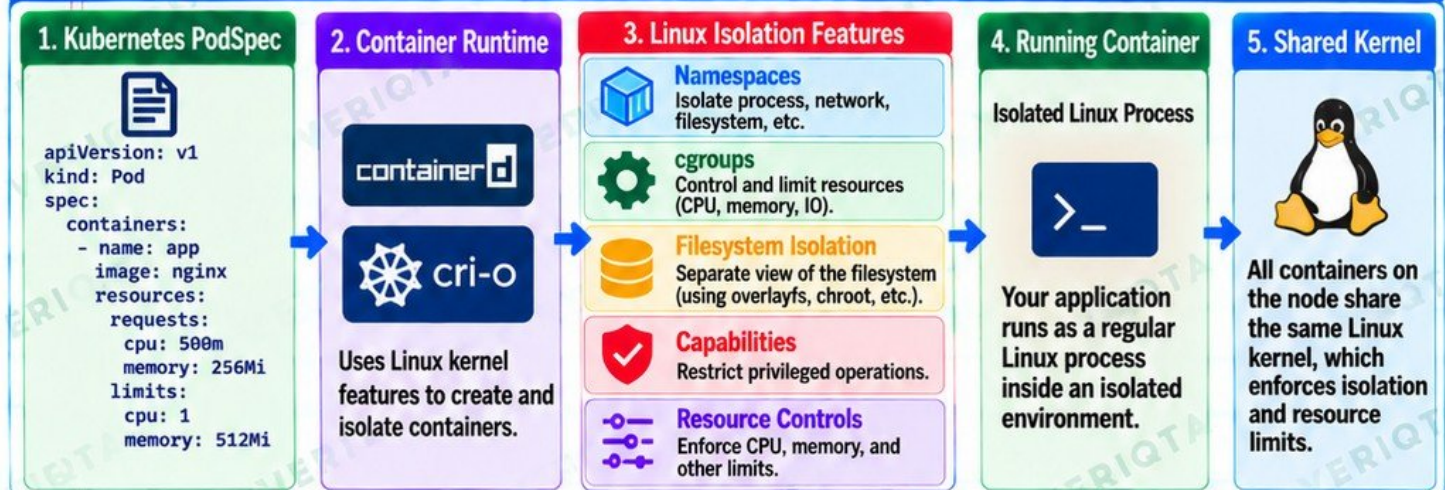


@veriqta

Containers Are Linux Processes

Containers are not virtual machines. They are isolated Linux processes that share the same kernel but are separated using namespaces, cgroups, filesystem isolation and other kernel features. Kubernetes uses these Linux capabilities (through container runtimes) to run and manage containers on each node.

From Kubernetes Configuration to a Running Linux Process



1 Linux Processes

- A container is a group of one or more Linux processes.
- The main application inside the container runs as a normal Linux process (PID 1).
- It is isolated from other processes using kernel features.
- When the container stops, the processes inside it stop.

2 Namespaces (Isolation)

- PID namespace: separate process tree (container only sees its own processes).
- Network namespace: isolated network stack (own interfaces, ports, routing).
- Mount namespace: isolated filesystem view.
- IPC namespace: isolated inter-process communication.
- UTS namespace: separate hostname and domain name.
- User namespace: maps container users to non-root users on the host (optional).

3 cgroups (Resource Control)

- Limit and track resource usage for container processes.
- CPU: limits the amount of CPU a container can use.
- Memory: limits the amount of memory and can trigger OOM kill if exceeded.
- Block IO: controls disk IO.
- Network IO: can limit network bandwidth.
- Kubernetes CPU and memory requests/limits are enforced using cgroups on the node.

4 Filesystem Isolation

- Containers get their own filesystem view (using technologies like overlays).
- Your application sees its own root filesystem (/) inside the container.
- Files from the container image are read-only layers, with a writable layer on top.
- Volumes can be mounted from the host or external storage into the container.

5 Capabilities (Permissions)

- Containers run with limited Linux capabilities by default.
- Capabilities control what privileged operations a process can perform (e.g. NET_ADMIN, SYS_ADMIN).
- This reduces the risk if a container is compromised.
- You can add or drop capabilities in the PodSpec.

6 From Kubernetes to Kernel

- You define CPU and memory requests/limits in the PodSpec.
- The kubelet and container runtime configure cgroups on the node.
- The container runtime uses namespaces, cgroups and other kernel features to create an isolated environment.
- Your application runs as a normal Linux process inside this isolated environment.

7 Why This Works

- All containers share the same Linux kernel.
- Namespaces isolate what containers can see.
- cgroups limit the resources containers can use.
- Filesystem isolation gives each container its own view of the filesystem.
- Capabilities and security features reduce the impact of compromised containers.

8 Key Takeaways

- ✓ Containers are isolated Linux processes, not virtual machines.
- ✓ Isolation is provided by namespaces, cgroups, filesystem isolation and capabilities.
- ✓ Kubernetes uses these Linux features (via container runtimes) to run containers.
- ✓ CPU and memory configuration in Kubernetes is enforced on the node using cgroups.
- ✓ All containers on a node share the same Linux kernel.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

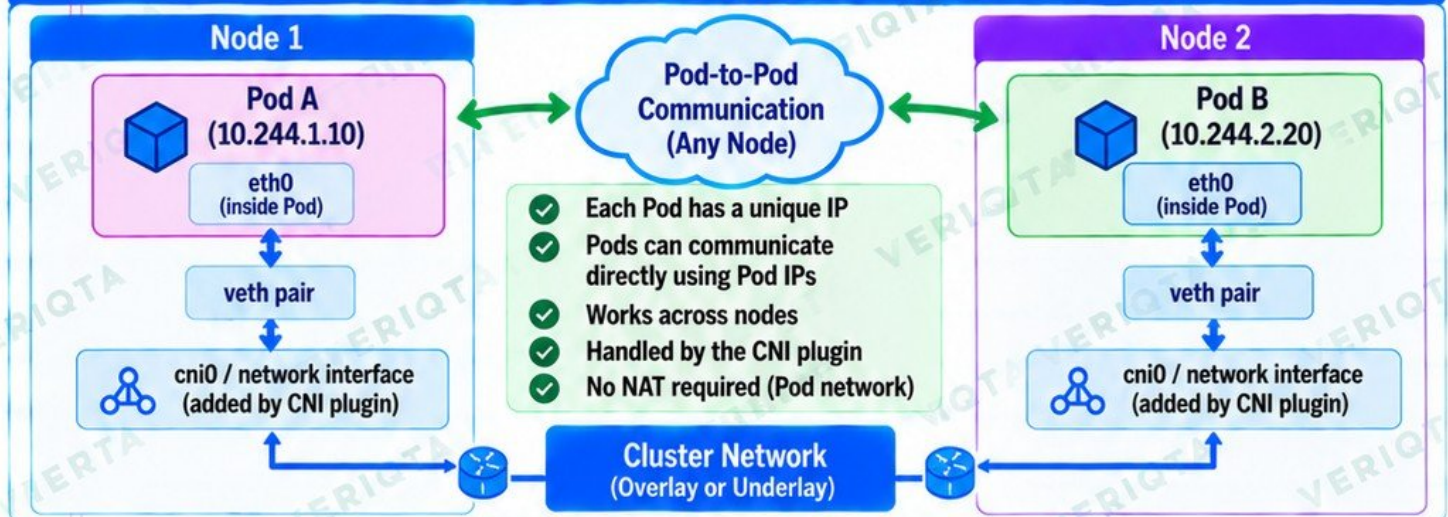


@veriqta

How Kubernetes Networking Actually Works

Kubernetes provides a network where every Pod gets a unique IP address and can communicate with any other Pod in the cluster, regardless of which node it is running on. This is achieved using Linux networking features and a CNI (Container Network Interface) plugin.

High Level View: Pod to Pod Communication



1 Unique Pod IPs

- Every Pod gets a unique IP address from the cluster Pod network.
- Pods use this IP to communicate directly with other Pods.
- The IP is assigned by the CNI plugin during Pod creation.
- Pods keep the same IP for their lifetime (when not recreated).

2 Network Namespaces

- Each Pod gets its own network namespace (isolated network stack).
- The containers in the Pod share the same network namespace.
- They see the same network interfaces, IP address and routes.
- Network namespaces provide isolation between Pods.

3 veth Pairs (Conceptually)

- A veth pair is a virtual Ethernet device.
- One end (eth0) is inside the Pod network namespace.
- The other end is on the node (connected to a bridge or interface created by the CNI plugin).
- It allows network traffic to move between the Pod and the node network.

4 Node Networking

- The CNI plugin creates network interfaces, bridges or routes on the node.
- It connects the Pod to the node network.
- Traffic from the Pod is routed to other Pods (on the same or different nodes) using the cluster network.
- Each node is configured by the CNI plugin to handle Pod traffic.

5 Pod-to-Pod Communication

- Pods can communicate directly using their Pod IPs.
- This works across nodes, not just on the same node.
- Kubernetes expects that all Pods can reach each other without NAT (using the Pod network).
- The CNI plugin ensures the routing, encapsulation or underlay setup needed for this.

6 The CNI Plugin's Role

- The CNI plugin is called during Pod creation (by kubelet).
- It sets up the Pod's network namespace, assigns a Pod IP and configures networking (veth pair, routes, etc).
- It also handles cleanup when the Pod is deleted.
- Different CNI plugins (e.g. Calico, Flannel, Cilium) use different methods (overlay, underlay, routing, eBPF, etc).

7 Routing Across Nodes

- Traffic between Pods on different nodes is routed through the cluster network.
- The CNI plugin configures the necessary routes, tunnels or underlay networking.
- Nodes know how to reach Pod IPs on other nodes via the cluster network.

8 Kubernetes Network Model

- Every Pod has a unique IP address.
- Every Pod can communicate with any other Pod without NAT.
- The Pod's network is flat (one network).
- Kubernetes does not prescribe how it is implemented — this is done by the CNI plugin.

9 Key Takeaways

- ✓ Each Pod has a unique IP and its own network namespace.
- ✓ Containers in a Pod share the network.
- ✓ The CNI plugin sets up networking during Pod creation.
- ✓ Pods can communicate directly across nodes.
- ✓ Different CNI plugins implement the network in different ways (overlay, underlay, eBPF, etc).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

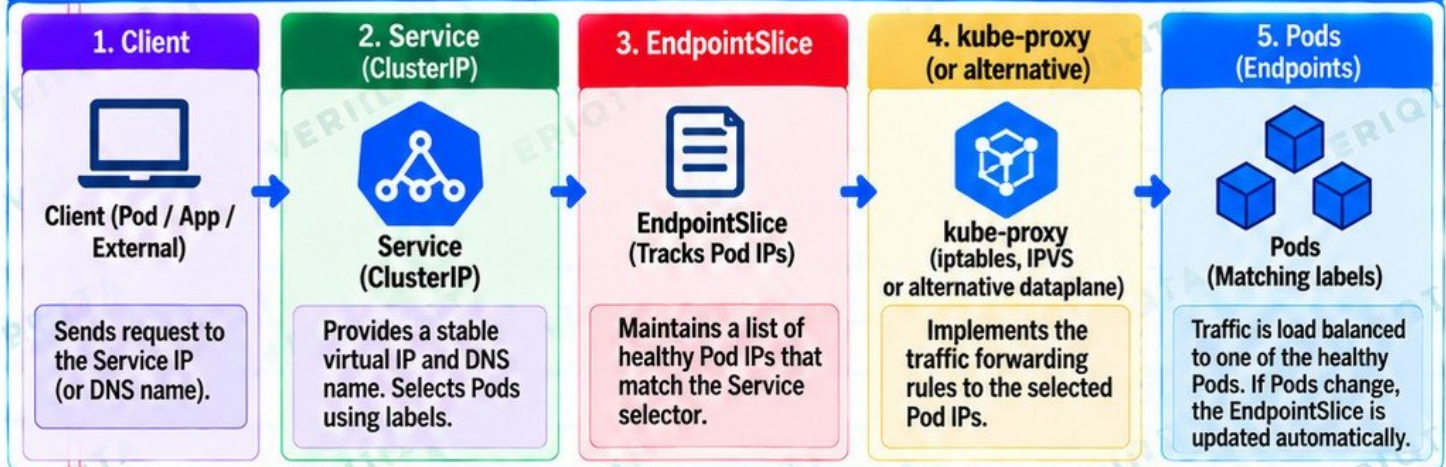


@veriqta

Services: How Traffic Finds Changing Pods

Pods have unique IP addresses, but these IPs can change when Pods are recreated. Services provide a stable virtual IP (ClusterIP) and DNS name, and automatically forward traffic to the healthy Pods using selectors and EndpointSlices.

How a Service Routes Traffic to Pods



1 Why Pod IPs Are Not Enough

- Pod IPs change when Pods are recreated.
- Pods are ephemeral and can come and go at any time.
- Clients should not depend on individual Pod IPs.
- Services provide a stable IP and DNS name that does not change even when Pods change.

2 Service Selectors

- A Service uses a label selector to find the Pods it should send traffic to.
- When Pods with matching labels are created or deleted, the Service automatically includes or removes them from the endpoints.
- This is fully automated and does not require manual updates.

3 EndpointSlices

- EndpointSlice stores the list of Pod IPs and ports for a Service.
- It replaces the older Endpoints resource and is more scalable.
- It is updated automatically when Pods that match the Service selector change (added, removed or become unhealthy).
- It helps Kubernetes handle large numbers of Pods efficiently.

4 Service Types (Focus on ClusterIP)

- ClusterIP:** Internal virtual IP for in-cluster communication (default).
- NodePort:** Exposes the Service on each node's IP and a static port.
- LoadBalancer:** Uses a cloud provider's load balancer to expose the Service externally.
- ExternalName:** Maps a Service to an external DNS name.

5 Traffic Forwarding (kube-proxy)

- kube-proxy watches Services and EndpointSlices.
- It creates network rules to forward traffic from the Service IP to one of the selected Pod IPs.
- It can use iptables mode, IPVS mode or an alternative dataplane (e.g. Cilium, eBPF).
- It provides load balancing across healthy Pods.

6 ClusterIP and DNS

- Each Service gets a stable ClusterIP (e.g. 10.96.0.10).
- It also gets a DNS name (e.g. my-service.default.svc.cluster.local).
- Pods can reach the Service using the ClusterIP or the DNS name.
- The DNS name is usually preferred because it is easier to remember and works across namespaces.

7 How Endpoints Stay Updated

- When a new Pod matching the selector is created, it is added to the EndpointSlice.
- When a Pod is deleted, it is removed from the EndpointSlice.
- If a Pod becomes unhealthy, it is also removed from the list of endpoints.
- This ensures traffic is only sent to healthy Pods.

8 Alternative Dataplane Implementations

- kube-proxy (iptables): default and simple.
- kube-proxy (IPVS): better performance for large clusters.
- Cilium (eBPF): uses eBPF instead of iptables/IPVS for faster and more flexible networking.
- All of these implementations provide the same core functionality: forward traffic from the Service to the selected Pods.

9 Key Takeaways

- Services provide a stable IP and DNS name for changing Pods.
- They use label selectors and EndpointSlices.
- kube-proxy or an alternative dataplane forwards traffic to healthy Pods.
- Pod IPs are not suitable for direct client use because they can change.
- This design enables reliable, scalable and dynamic communication in Kubernetes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

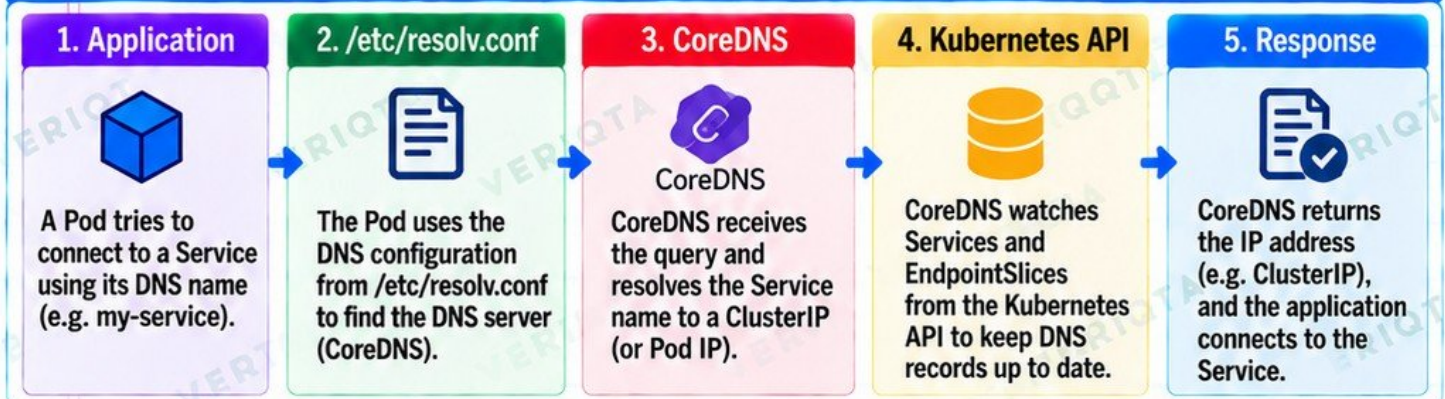


@veriqta

DNS and Service Discovery Inside the Cluster

Kubernetes uses CoreDNS to provide DNS for Services and Pods inside the cluster. This allows applications to discover and communicate with other Services using stable DNS names, without needing to know Pod IPs.

How DNS Resolution Works in Kubernetes



1 Service DNS Names

- Every Service gets a DNS name in the format:
`<service-name>.<namespace>.svc.cluster.local`
- You can also use shorter names from within the same namespace, e.g.
`my-service`
- Or include the namespace:
`my-service.my-namespace`

2 /etc/resolv.conf in a Pod

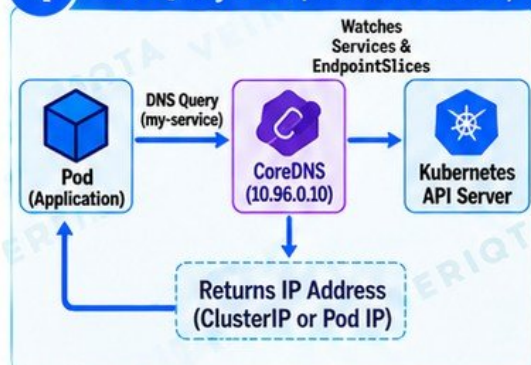
- Every Pod gets DNS configuration automatically.
- It points to the CoreDNS Service IP (as the nameserver).
- It includes search domains, allowing short names to work.

```
# Example /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local
svc.cluster.local
cluster.local
options ndots:5
```

3 Search Domains (Namespace Aware)

- Search domains allow you to use short service names.
- If you run `nslookup my-service` in the default namespace, Kubernetes will try:
 - `my-service.default.svc.cluster.local`
 - `my-service.svc.cluster.local`
 - `my-service.cluster.local`
- This makes it easy for applications to find Services in the same namespace without using the full DNS name.

4 DNS Query Flow (Inside the Cluster)



5 Useful DNS Commands

- nslookup**
 - Quickly resolve a DNS name.
 - Shows the IP address.

```
nslookup my-service
```
- dig**
 - Provides detailed DNS information.
 - Useful for troubleshooting.

```
dig my-service.default.svc.cluster.local
```
- getent**
 - Uses the system resolver.
 - Simple and commonly available.

```
getent hosts my-service
```

6 Why DNS Matters

- Pod IPs are ephemeral and can change when Pods are recreated.
- DNS provides a stable name for Services.
- Applications can discover and connect to other Services without hardcoding IP addresses.
- If DNS fails, applications may report "cannot be found" or "name resolution errors".
- Tools like `nslookup` and `dig` help determine if the issue is DNS, the Service, or the network.

7 Common Issues and Troubleshooting

- Cannot resolve service name → Check CoreDNS Pods are running.
- Timeout or no response → Check network policies and DNS Service IP.
- Wrong IP returned → Check the Service and EndpointSlice.
- Intermittent failures → Check CoreDNS logs and resource usage.
- Use `nslookup` or `dig` to verify the DNS name and IP.
- Ensure you are using the correct namespace or full DNS name.

8 Key Takeaways

- CoreDNS provides DNS for Services and Pods inside the cluster.
- Services get stable DNS names in the format `<service>.<namespace>.svc.cluster.local`.
- `/etc/resolv.conf` in Pods points to the CoreDNS Service IP with appropriate search domains.
- DNS resolution is namespace-aware, allowing short names to work.
- `nslookup`, `dig` and `getent` are essential tools for troubleshooting.
- DNS enables reliable service discovery even when Pod IPs change.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

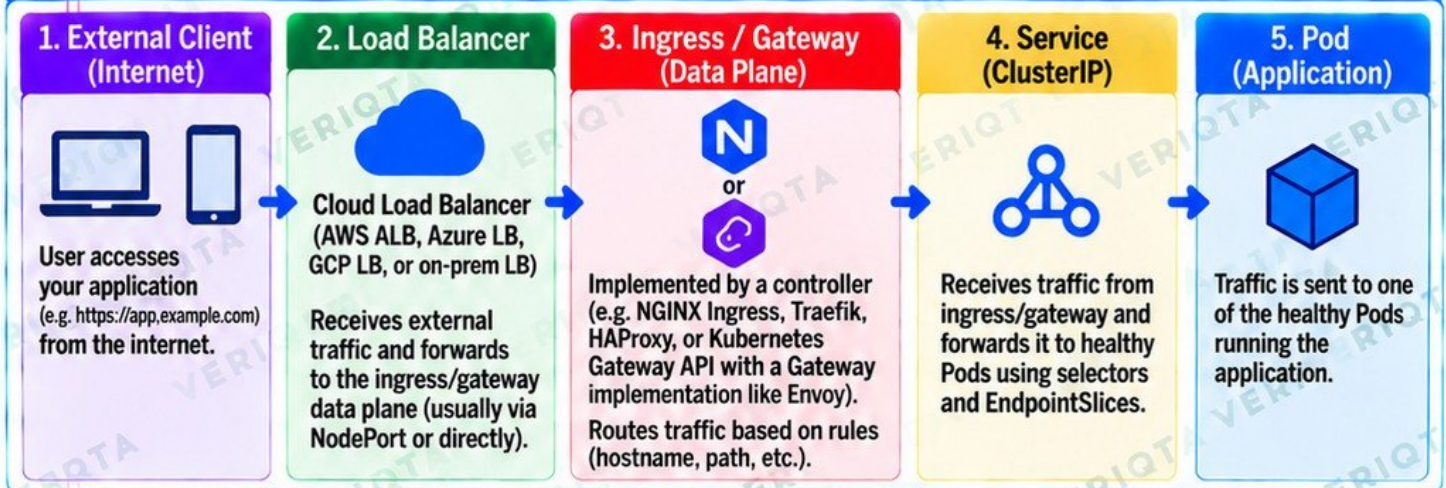


@veriqta

Ingress, Gateway and North-South Traffic

Ingress and Gateway allow external (north-south) traffic to reach Services inside the cluster. They require a controller and data plane implementation to actually route traffic. This traffic then goes to a Service and finally to the Pods.

End-to-End Flow: External Traffic to a Pod



1 What is Ingress?

- An Ingress is a Kubernetes API object that defines routing rules (host, path, etc.).
- On its own, an Ingress object does NOT route any real network traffic.
- It requires a controller and a data plane (e.g. NGINX Ingress, Traefik, HAProxy) to watch the object and configure proxies that route traffic.
- The controller reads the Ingress rules and updates the data plane with the correct configuration.

2 What is a Gateway?

- Gateway is the next-generation API (Gateway API).
- It provides more advanced and flexible traffic management than Ingress (e.g. HTTP, TCP, TLS, advanced routing).
- It separates responsibilities using Gateway, GatewayClass, and HTTPRoute.
- It also requires a controller and a data plane (e.g. Envoy, NGINX, or cloud provider implementations) to actually route traffic.

3 North-South vs East-West

- **North-South traffic:** traffic that comes from outside the cluster (internet or external network) into the cluster. Handled by Load Balancers, Ingress or Gateway.
- **East-West traffic:** traffic between services and Pods inside the cluster. Usually handled by Services (ClusterIP) and the cluster network (CNI).
- Ingress/Gateway are for north-south traffic. Services are for east-west traffic (and also the target for north-south).

4 Ingress vs Gateway

- **Ingress:** older, simpler API, mainly HTTP/HTTPS routing.
- **Gateway:** newer, more powerful and flexible (supports multiple protocols, advanced routing, better extensibility).
- Both serve the same purpose: expose applications to external traffic.
- Both require a controller and a data plane implementation.

5 Common Implementations

- NGINX Ingress Controller (Ingress).
- Traefik (Ingress).
- HAProxy Ingress.
- Kubernetes Gateway API with Envoy (Gateway).
- Cloud provider ingress/gateway (e.g. AWS ALB Ingress Controller, Azure Application Gateway, GCP HTTP(S) Load Balancer).

6 Key Points to Remember

- An Ingress object alone does not magically route packets.
- You must have an implementation (controller + data plane).
- External traffic usually first hits a Load Balancer, then the ingress/gateway, then a Service, then a Pod.
- Ingress/Gateway use Services as the backend, not Pods directly.
- Services use selectors and EndpointSlices to find healthy Pods.

7 Example Ingress Flow (NGINX)



8 Key Takeaways

- ✓ Ingress and Gateway expose services to external (north-south) traffic.
- ✓ They require a controller and data plane to work.
- ✓ Load balancer sends traffic to the ingress/gateway.
- ✓ Ingress/gateway routes traffic to a Service.
- ✓ Service forwards traffic to healthy Pods using EndpointSlices.
- ✓ North-south is external traffic, east-west is internal cluster traffic.
- ✓ Choose Ingress for simple HTTP/HTTPS, or Gateway for more advanced use cases.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

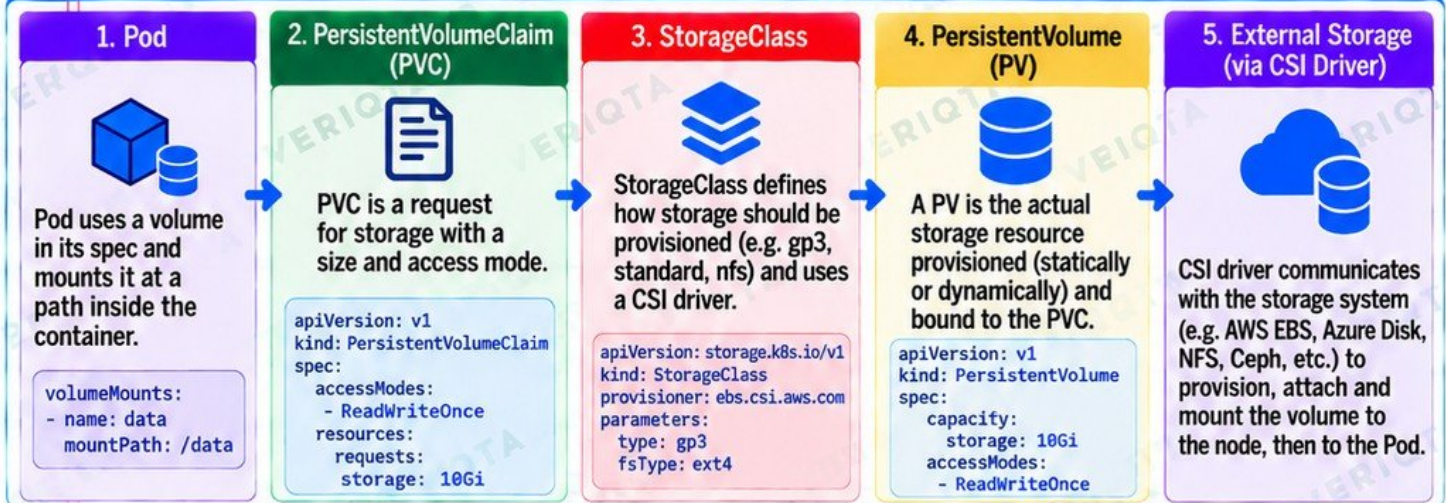


@veriqta

Storage Under the Hood

Kubernetes provides storage for stateful applications using volumes, PersistentVolumes (PVs), PersistentVolumeClaims (PVCs) and StorageClasses. CSI (Container Storage Interface) drivers integrate Kubernetes with different storage systems and enable dynamic provisioning.

How Kubernetes Storage Works



1 Pod Volumes

- A Pod can use different types of volumes (emptyDir, configMap, secret, persistent volume, etc.).
- Volumes are mounted into the container at a specific path.
- emptyDir and container filesystem are temporary and are deleted when the Pod is removed.
- Use PersistentVolumes for data that must survive Pod restarts or rescheduling.

2 PersistentVolume (PV)

- PV is a cluster resource that represents actual storage in the cluster or external system.
- It can be created statically or dynamically.
- It has a capacity, access mode and a storage class.
- A PV is bound to a PVC before it can be used by a Pod.

3 PersistentVolumeClaim (PVC)

- PVC is a request for storage made by a user.
- You specify the required size and access mode.
- Kubernetes finds a matching PV, or dynamically provisions one using a StorageClass.
- When the PVC is bound, the volume is mounted into the Pod.

4 StorageClass

- StorageClass defines how storage should be provisioned (which CSI driver to use and with what parameters).
- It enables dynamic provisioning.
- Different StorageClasses can be used for different performance or cost needs (e.g. gp3, standard, fast-ssd, nfs).
- If no StorageClass is specified, the default StorageClass (if any) will be used.

5 CSI Drivers

- CSI (Container Storage Interface) allows Kubernetes to work with many storage systems.
- The CSI driver handles provisioning, attaching, mounting and unmounting volumes.
- Examples: AWS EBS CSI, Azure Disk CSI, GCE PD CSI, NFS CSI, Ceph CSI, vSphere CSI.
- CSI drivers enable dynamic provisioning based on the StorageClass.

6 Attach and Mount Process

- For some storage types (e.g. EBS, PD), the volume is attached to the node first.
- The kubelet then mounts the volume to the node filesystem.
- Finally, it is bind-mounted into the container at the specified path.
- For network storage (e.g. NFS), the volume is mounted directly using the CSI driver on the node.

7 Why Container Filesystem Is Different

- The container filesystem (image + writable layer) exists only for the lifetime of the Pod.
- When the Pod is deleted, the container filesystem is lost.
- Application data that must persist (e.g. databases, user files) should be stored on a PersistentVolume.
- This allows data to survive Pod restarts, rescheduling or node failures.

8 Common Pending PVC / Volume Errors

- No matching StorageClass.
- StorageClass exists but cannot provision a volume (e.g. quota, permission, wrong parameters).
- Insufficient storage capacity.
- CSI driver not installed or not working.
- Volume attachment or mount failure on the node.
- Wrong access mode (e.g. ReadWriteOnce used by multiple nodes).
- Node does not have the required permissions.

9 Key Takeaways

- Use PVCs and StorageClasses for persistent data.
- CSI drivers integrate Kubernetes with storage systems.
- Dynamic provisioning simplifies storage management.
- Container filesystems are temporary.
- Check events and describe commands when a PVC is Pending or a volume fails to mount.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



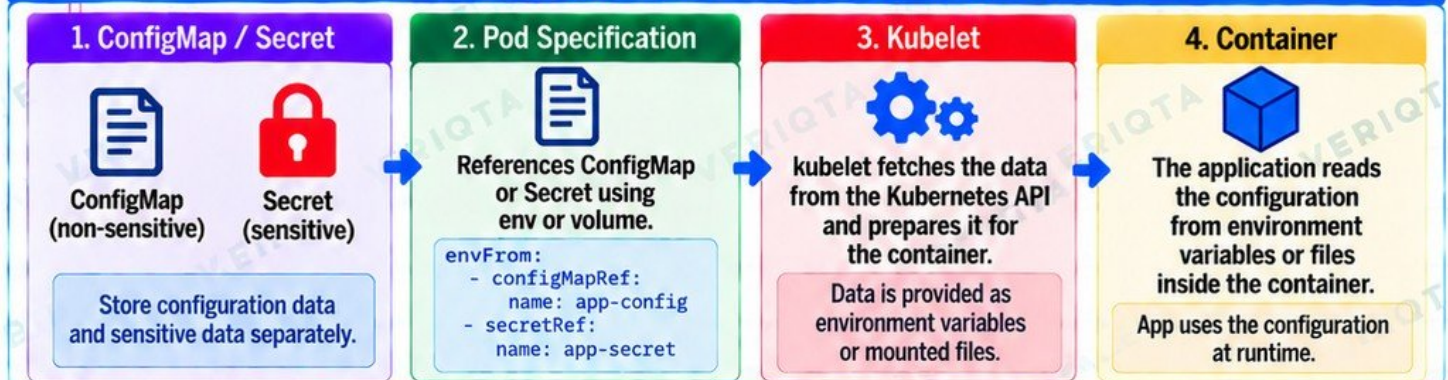
@veriqta

ConfigMaps, Secrets and Workload Configuration

ConfigMaps and Secrets allow you to provide configuration to containers without baking it into the image. They can be consumed as environment variables or mounted as files.

Secrets store sensitive data (base64 encoded, not encrypted) and should be protected with proper access controls and best practices.

How Configuration Reaches Containers



1 What is a ConfigMap?

- Used to store non-sensitive configuration data (e.g. app settings, feature flags, URLs).
- Can be consumed as environment variables or mounted as files.
- Useful for separating configuration from container images.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
  API_URL: https://api.example.com
```

2 What is a Secret?

- Used to store sensitive data such as passwords, API keys, tokens and TLS certificates.
- Data is base64 encoded, not encrypted.
- Should be protected with RBAC, least privilege and proper access controls.
- Avoid storing sensitive data in container images or Git repositories.

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: bXlyYXNzd29yZA==
  API_KEY: YWJMTlZNDU=
```

3 Consuming as Environment Variables

- Useful for simple key value configuration.
- Values are available as environment variables inside the container.
- Pods need to be restarted to pick up changes to environment variables.

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: app-secret
        key: DB_PASSWORD
  - name: APP_ENV
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_ENV
```

4 Consuming as Mounted Files

- Configuration is mounted as files inside the container.
- Useful for larger configuration files (e.g. application.conf, nginx.conf).
- Files are mounted as read-only by default.
- Changes are updated automatically on the node, but the application may need a reload to pick up new values.

```
volumeMounts:
  - name: config-volume
    mountPath: /etc/app/config
    readOnly: true
volumes:
  - name: config-volume
    configMap:
      name: app-config
```

5 Updates and Reload Behavior

- Mounted ConfigMaps and Secrets are updated automatically on the node (usually within a few seconds).
- The application may need to reload (you can send a SIGHUP or use a sidecar/reloader) to apply changes.
- Environment variables are not updated automatically. A Pod restart is required.
- Design applications to handle configuration reloads gracefully.

Reload Options:

- Application supports SIGHUP.
- Use a reloader sidecar (e.g. stakater/reloader).
- Perform a rolling restart of the Deployment.

6 Security and Best Practices

- Never bake credentials into container images.
- Use Secrets for sensitive data and ConfigMaps for non-sensitive data.
- Use RBAC to restrict who can access Secrets.
- Use least privilege (only give access to what is needed).
- Consider using external secret managers (e.g. AWS Secrets Manager, Azure Key Vault, HashiCorp Vault) for better security.
- Avoid logging sensitive values.

7 Common Issues and Troubleshooting

- Secret or ConfigMap not found.
- Incorrect key name in env or volume.
- Permission denied (RBAC).
- Application not reloading after changes.
- Pod crash due to missing configuration.
- Check events and logs: `kubectl describe pod ...`

8 Why Not Bake into the Image?

- Images are immutable and may be shared publicly.
- Credentials in images can be extracted by anyone.
- Separating configuration from images allows flexibility across environments (dev, staging, prod).
- Using ConfigMaps and Secrets follows the 12-factor app principle.

9 Key Takeaways

- Use ConfigMaps for non-sensitive data.
- Use Secrets for sensitive data (base64 is not encryption).
- Provide configuration via environment variables or mounted files.
- Know how updates and reloads work.
- Follow least privilege and protect Secrets.
- Do not bake credentials into container images.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

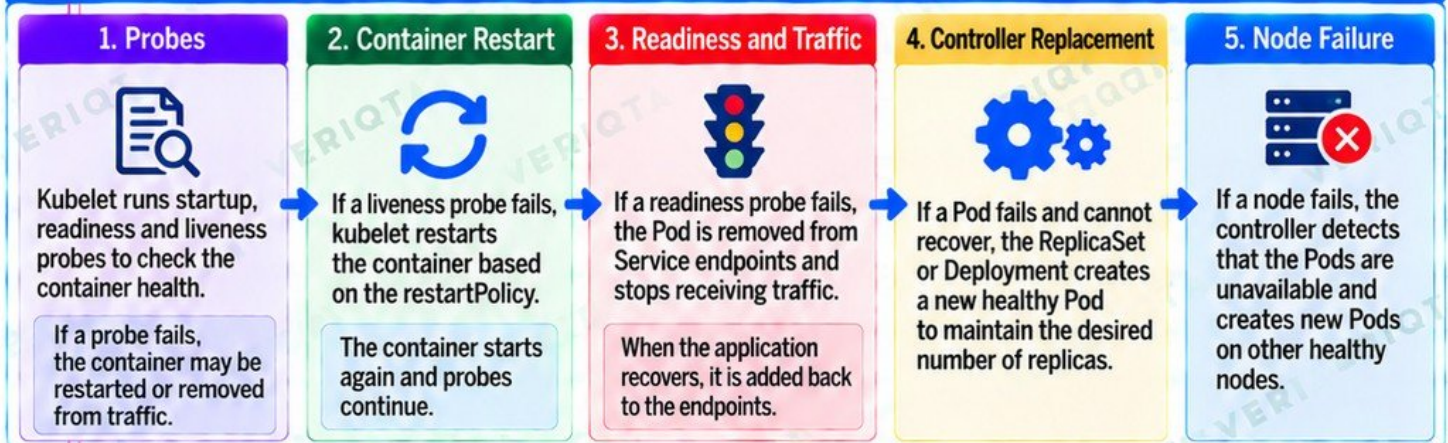


@veriqta

Health Checks, Restarts and Self-Healing

Kubernetes uses health checks, restart policies and controllers to keep applications running. Probes determine when a container is healthy, ready to receive traffic, or needs to be restarted. Controllers like Deployments and ReplicaSets replace failed Pods and handle node failures.

How Health Checks and Self-Healing Work Together



1 Types of Probes

- **Startup Probe**
Checks if the application has started. Used for slow starting applications.
- **Readiness Probe**
Checks if the application is ready to receive traffic.
- **Liveness Probe**
Checks if the application is still running. If it fails, the container is restarted.

2 Probe Configuration Example

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
    initialDelaySeconds: 30
    periodSeconds: 10
    failureThreshold: 3
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10
    failureThreshold: 3
```

3 Container Restart Behavior

- When a liveness probe fails, kubelet kills the container and starts it again.
- The restartPolicy controls how containers are restarted (Always, OnFailure, Never).
- Restart count increases each time the container restarts.
- If the container keeps failing, Kubernetes continues to restart it based on the policy.

4 Readiness and Traffic Handling

- When a readiness probe fails, the Pod is removed from Service endpoints (EndpointSlices).
- The Pod stops receiving new traffic but continues running.
- When the probe succeeds again, the Pod is added back to the endpoints.
- This prevents sending traffic to an unhealthy application.

5 Controller Self-Healing

- Deployments and ReplicaSets ensure the desired number of healthy Pods are running.
- If a Pod is deleted, crashes or becomes unhealthy, a new Pod is created.
- They continuously monitor Pod status and take action to replace failed Pods.
- This is different from container restarts which are handled by kubelet.

6 Node Failure Behavior

- If a node becomes unreachable, Pods on that node are marked as unavailable.
- The controller creates new Pods on other healthy nodes.
- Once the original node recovers, the Pods may be recreated there depending on the controller and the cluster configuration.
- This helps maintain application availability even during node failures.

7 restartPolicy Options

Policy	Behavior
Always	Always restart the container (if it exits). Used for most applications.
OnFailure	Restart only when the container exits with a non zero status.
Never	Do not restart the container. Commonly used for Jobs.

8 Key Mechanisms in Self-Healing

- ✓ Probes detect unhealthy or unready containers.
- ✓ Container restarts are handled by kubelet.
- ✓ Readiness removes unhealthy Pods from traffic.
- ✓ Controllers replace failed or missing Pods.
- ✓ Node failure handling reschedules Pods to other nodes.
- ✓ All these mechanisms work together to provide self-healing, there is no single feature that does it all.

9 Common Issues and Troubleshooting

- ✓ Liveness probe fails repeatedly
→ check application logs and probe configuration.
- ✓ Readiness probe never succeeds
→ check application startup, dependencies and probe path.
- ✓ Pod keeps restarting
→ check logs, resource limits and application errors.
- ✓ Pod not replaced after failure
→ check controller status and events.
- ✓ Node failure not handled as expected
→ check node status and controller events.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

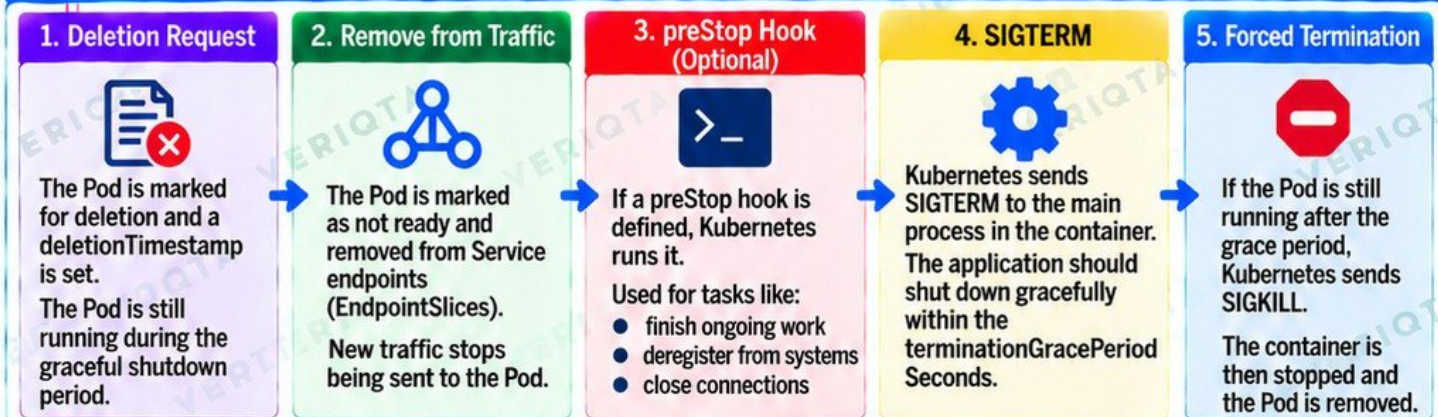


@veriqta

Pod Termination, Updates and Failure Paths

Pods can terminate intentionally or unexpectedly. Kubernetes handles graceful shutdown, removal from traffic, and replacement using several mechanisms. Understanding the termination flow, update process and common failure paths helps you design reliable applications.

1. What Happens When a Pod is Terminated?



2 Key Fields for Graceful Shutdown

- preStop**
A hook that runs before the container receives SIGTERM.

```
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 10"]
```
- terminationGracePeriodSeconds**
Time given for the container to shut down gracefully after SIGTERM (default 30 seconds).

```
terminationGracePeriodSeconds: 60
```

3 Rolling Updates (Deployments)

- A new Pod is created with the new version.
- The new Pod becomes ready.
- Traffic is shifted to the new Pod.
- The old Pod is terminated using the graceful shutdown process (`preStop` → SIGTERM → grace period → SIGKILL if needed).

Update Strategy Options:

- RollingUpdate (default)**
 - `maxSurge`: extra Pods above desired count
 - `maxUnavailable`: number of Pods that can be down during the update
- Recreate**
 - All old Pods are terminated before new ones are created (causes downtime).

4 Replacement and Recovery

- Deployments/ReplicaSets automatically create a new Pod when one is terminated.
- The new Pod is scheduled on a healthy node.
- Readiness probe must succeed before the new Pod receives traffic.
- If a node fails, the controller recreates the Pod on another node.
- This ensures the desired number of healthy Pods are always running.

Self-healing is achieved through:

- Readiness and liveness probes
- Controllers (Deployments, ReplicaSets)
- Node failure handling
- Automatic rescheduling

5 Common Failure Paths

Failure	Cause and Explanation
OOMKilled	Container uses more memory than its limit. The kernel kills the process.
CrashLoopBackOff	Container keeps crashing and restarting. Often caused by application errors or misconfiguration.
Error	Container exits with a non zero exit code.
Evicted	Pod is evicted due to node pressure (CPU, memory, disk) or other conditions.
Node Not Ready	Node becomes unreachable. Pods are rescheduled to other healthy nodes.

6 Example: Graceful Shutdown

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: app
          image: my-app:1.0
          lifecycle:
            preStop:
              exec:
                command: ["/bin/sh", "-c", "sleep 10"]
          ports:
            - containerPort: 8080
```

7 Key Takeaways

- ✓ Pod termination starts with a `deletionTimestamp`.
- ✓ Pod is removed from Service endpoints before shutdown.
- ✓ Use `preStop` and a proper grace period for graceful shutdown.
- ✓ Kubernetes sends SIGTERM, then SIGKILL if needed.
- ✓ Controllers automatically replace terminated Pods.
- ✓ Rolling updates ensure zero or minimal downtime.
- ✓ Understand common failure states like OOMKilled, CrashLoopBackOff and Evicted.
- ✓ Design applications to handle termination signals and shut down gracefully.

8 Troubleshooting Commands

<code>kubectl get pods</code>	Check Pod status
<code>kubectl describe pod <pod></code>	See events and termination reason
<code>kubectl logs <pod></code>	Check application logs
<code>kubectl get events --sort-by=.lastTimestamp</code>	View recent events
<code>kubectl top pod <pod></code>	Check resource usage

9 Pod Termination Flow (Diagram)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

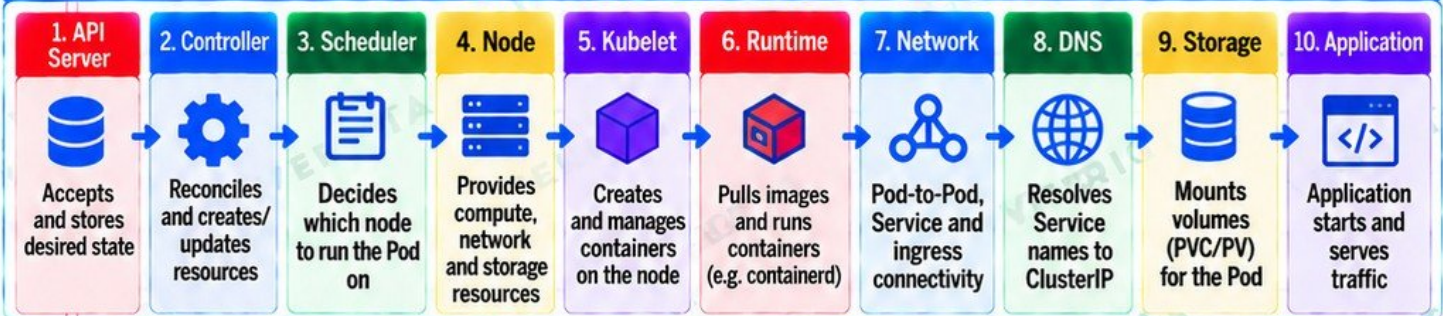


@veriqta

Debug Kubernetes From the Inside Out

Use a structured, evidence-first approach to troubleshoot Kubernetes problems. Follow the request or workload from the API to the application, checking each layer for issues instead of random restarts.

Kubernetes Components to Investigate (End-to-End Flow)



1 Start With the Symptoms

- What is not working? (Pod, Service, ingress, application)
- When did it start?
- What changed recently? (deploy, config, node, network, storage)
- Collect error messages and user impact information.

2 Check Kubernetes Resource Status

- Check the current state of your resources.
- ```
kubectl get pods -A
kubectl get deployments
kubectl get svc,ep,endpointslices
kubectl get ingress
kubectl get nodes
kubectl get events -A --sort-by=.lastTimestamp
```

### 3 Describe for More Details

- Shows detailed information, conditions, events and recent changes.
- ```
kubectl describe pod <pod>
kubectl describe deploy <name>
kubectl describe svc <name>
kubectl describe node <node>
kubectl describe pvc <name>
```

4 Check Events (What Happened?)

- Events show what the cluster is doing and why something failed.
- ```
kubectl get events -A --sort-by=.lastTimestamp
kubectl describe pod <pod> | grep -i event
```

### 5 Check Logs (Controller and Pod)

- Controller logs (for cluster components).
  - Application logs (for container issues).
- ```
kubectl logs <pod>
kubectl logs <pod> -c <container>
kubectl logs -n kube-system <controller-pod>
```

6 Exec Into the Pod

- Check inside the container to verify files, environment variables, network, DNS and application status.
- ```
kubectl exec -it <pod> -- /bin/bash
kubectl exec -it <pod> -- sh
kubectl exec <pod> -- env
kubectl exec <pod> -- cat /etc/resolv.conf
```

### 7 Check Resource Usage and Status

- Verify CPU, memory, restarts and limits.
  - Identify OOMKilled, CrashLoopBackOff or high resource usage.
- ```
kubectl top pods
kubectl top nodes
kubectl get pod <pod> -o wide
kubectl describe pod <pod>
```

8 Check Networking

- Test connectivity to Services, Pods and external endpoints.
 - Check if the application can reach other services.
- ```
kubectl exec -it <pod> -- ping <service>
kubectl exec -it <pod> -- curl http://<service>
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- dig <service>
```

### 9 Check DNS Resolution

- Verify that Service names resolve correctly.
  - Check /etc/resolv.conf and DNS search domains.
- ```
kubectl exec -it <pod> -- cat /etc/resolv.conf
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- dig <service>
kubectl exec -it <pod> -- getent hosts <service>
```

10 Check Storage

- Verify PVC, PV and StorageClass status.
 - Check if the volume is mounted and accessible inside the Pod.
- ```
kubectl get pvc,pv,storageclass
kubectl describe pvc <name>
kubectl exec -it <pod> -- df -h
kubectl exec -it <pod> -- ls -lah /mount/path
```

### 11 Check Node and Kubelet

- Verify node health, kubelet status and recent issues.
  - Look for disk pressure, memory pressure or kubelet errors.
- ```
kubectl get nodes -o wide
kubectl describe node <node>
kubectl get events -A | grep -i node
# On the node (if you have access)
sudo journalctl -u kubelet -n 100
```

12 Follow a Troubleshooting Flow

- Check API objects and events
- Check controller and deployment status
- Check scheduler and node placement
- Check node and kubelet
- Check container runtime and Pod status
- Check networking and DNS
- Check storage
- Check application logs and behavior

13 Common Failure Indicators

- Pending Pod → scheduling, resource or storage issue
- ImagePullBackOff → image or registry issue
- CrashLoopBackOff → application or config issue
- OOMKilled → memory limit too low
- Evicted → node pressure
- Service not reachable → selector, endpoints or network issue
- Cannot resolve name → DNS or CoreDNS issue
- Volume mount error → PVC, PV, StorageClass or CSI issue

14 Evidence-First Mindset

- Check facts before making changes
- Understand the real error message
- Follow the flow from API to application
- Use logs, events and status information
- Avoid random restarts
- Make small, targeted changes
- Verify the fix and monitor

15 Key Takeaways

- Kubernetes troubleshooting is a layered process.
- Use kubectl get, describe, events, logs and exec.
- Check each component from API to application.
- Use evidence, not assumptions.
- Understand common failure patterns.
- Fix the root cause, not just the symptom.
- This approach reduces downtime and builds stronger production systems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta